

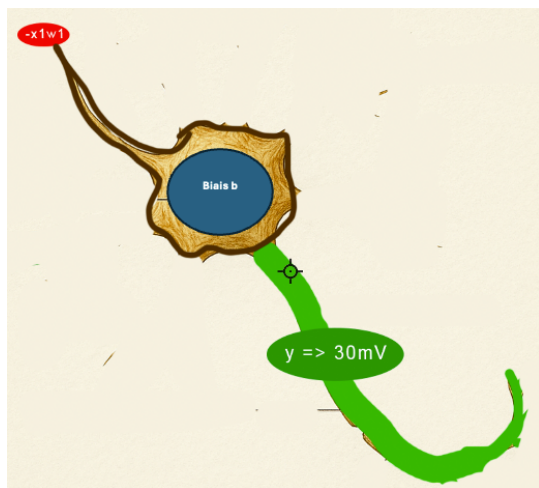


Prérequis avoir fini le TP 1 AI et avoir installé les outils python sur sa VM et VScode

TP 4 IA : Programmation d'un Perceptron

Exercice 1 : Chargement des données d'entrainement. ●

(NE PAS UTILISER WEB NI IA)



Pour entrainer une IA, il faut lui envoyer une quantité astronomique de data. Dans notre cas c'est un petit neurone Perceptron qui prédit un danger selon une vitesse.

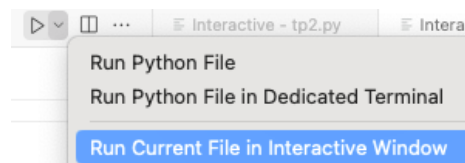
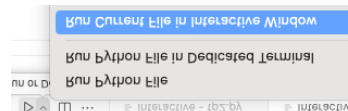
Pour charger des données en python on va utiliser la méthode `read_csv("NonFichier.csv")` de la librairie pandas.

Exercice 1 : dans un fichier TP4.py

Rappel , il n'y a pas de ";" pour terminer une instruction en python

- 1) Importé la librairie et nommé la **pandasLib**
`import pandas as nomDeLaLibDansVotreCode`

Vérifier que le code fonctionne en appuyant sur



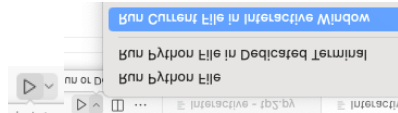
- 2) appeler la méthode `read_csv("donnevrai.csv")` sur l'objet pandasLib

Comme La méthode `read_csv` renvoi un objet de type `DataFrame`. Il faut appelé cette méthode derrière une variable et l'opérateur "="

Exemple pour vous aider : `maVariable = unObjetPanda.uneMethodedeLaLib(desArguments)`

Pour rendre lisible le code la variable qui va récupérer le `dataFrame` vous allez l'appeler non pas "df" mais `dataFrame1MonJeuDeDonneeDentrainement` =
`dataFrame1MonJeuDeDonneeDentrainement` =

Vérifier que le code fonctionne en appuyant sur



3) Si votre jeu de donnée possède en sortie quelque chose de différent que 0 ou 1 il faut adapter son jeu de donnée pour que le y attendu soit 0 et 1 et pas danger ou safe.

Pour cela on utilise une pirouette de programmation :

Encodage : danger → 1, safe → 0 adapter votre code à vos nom de variables
`dataFrame["danger"] = dataFrame["danger"].map({"safe": 0, "danger": 1})`

Demander à votre IA ou sur le Web de vous expliquer la méthode `.map`

4) Mettre notre jeu de donnée dans un tableau d'entrée X et de sortie attendu Y.

Cela va permettre d'avoir un tableau X avec toutes les valeurs $x_1 x_2 x_3 \dots x_n$ et un tableau Y qui aura toutes les valeurs $y_1_attendu, y_2_attendu \dots y_n_attendu$

Pour accéder au données $x_1 x_2 x_3$ (c'est-à-dire des entrées de notre jeu de donnée) on peut faire :

`dataFrame1MonJeuDeDonneeDentrainement["vitesse"]`

Pour accéder au données $y_1_attendu, y_2_attendu \dots y_n$ on peut faire :

`dataFrame1MonJeuDeDonneeDentrainement["vitesse"]`

vous devez tester vos données dans la fenetre interactive

dataFrame1MonJeuDeDonneeDentrainement["danger"]		
[5]	✓	0.0s
...	0	0
	1	1
	2	0
	3	1
	4	0
	5	1
	6	0
	7	1
	8	0
	9	1
	10	0
	11	1
	--	-
dataFrame1MonJeuDeDonneeDentrainement["danger"]		

actuellement nos données sont dans un `dataFrame` une structure que l'on ne maîtrise pas pour le moment. Mais on peut convertir un `dataFrame` en un tableau avec la méthode `.tolist()`

```
dataFrame1MonJeuDeDonneeDentrainement["danger"].tolist()
✓ 0.0s
[0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1]
```

Donc pour répondre à la question on peut faire X égale leDataFrame de la vitesse .tolist et idem pour Y sur le dataFrame des danger.

Vérifiez que votre code fonctionne avec l'expression "print" suivante :

Exercice 2 : Initialisation 🟡

Ne pas utiliser IA

Rajouter dans votre code python les lignes qui initialise une variable w et b avec une valeur aléatoire entière comprise entre -10 et 10.

```
import random
```

utilise la méthode de de l'objet random que tu viens d' importé et qui s'appel

```
randint(valeurMin, valeurMax)
```

rappel en POO Programmation Orienté Objet pour appeler une méthode d'un objet on fait

```
objet.methode(...)
```

Puis afficher ces 2 variables dans un print().

```
print(X[:5], Y[:5])
✓ 0.0s
[32.0, 85.4, 73.0, 91.0, 36.2] [0, 1, 0, 1, 0]
```

Demander à votre IA ou sur le Web de vous expliquer la syntaxe NomTableau[:nombre] sur un tableau en python. Maitriser les astuces de code renforce votre niveau en programmation.

Exercice 3 : Propagation avant (forwarding)

Rappel : Dans la phase de propagation avant, notre neurone calcule une **combinaison linéaire** des entrées, sous la forme $Z = w1*x1+B$.

Le but de cet exercice 3 est d'apprendre à définir une fonction en python dans un premier temps. Puis dans un deuxième temps, d'écrire la fonction $Z_{predict}(x)$ qui applique la fonction mathématique vu dans la leçon 2 : $Z_{predict} = w * x + b$

1) Définir une fonction qui prend un paramètre fait un calcul et retourne un résultat.

Voici la syntaxe :

```
def nomDeLaFonction(Paramètres1, Paramètre2 ect):  
    calcul = Paramètres1 * Paramètres2  
    return calcul
```

donc la définition commence par le mot clé **def** et la ligne se finit par **":"**

ensuite tout son code est décalé d'une tabulation .

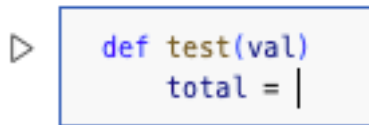
Attention le **return** lui est aussi est décalé d'une indentation par rapport au mot clé **def**

Dans la fenêtre interactive uniquement pour ne pas polluer votre code

Écrire une fonction test qui prend en paramètre un seul paramètre

Et qui fait comme calcul la chose suivante : le paramètre se multiplie avec lui-même .

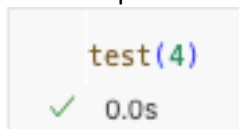
Puis retourner le résultat.



```
def test(val)  
    total = |
```

Appuyer sur la flèche pour valider votre fonction

Une fois la méthode défini. Toujours dans la fenetre interactive, Appeler votre méthode et tester une valeur de paramètre qui vaut 4



```
test(4)  
✓ 0.0s
```

16

Si votre fonction test retourne 16 alors vous avez réussi à créer une méthode en python.

2) Définir la fonction de prediction Z de votre neurone

Dans votre fichier TP4.py définition la fonction $Z_{predict}$

Aide :

nom de la fonction : " $Z_{predict}$ " ,

paramètre de la fonction : x

calcul attendu : $w*x+b$

Sortie attendu : si $Z \geq 0$ alors sortie 1 sinon sortie 0

Valider votre définition de Zpredict en exécutant TP4.py  sinon elle ne sera pas prise en compte.

Une fois la fonction définie, retournez dans votre fenêtre interactive tester la boucle suivante

```
for x in X:  
    print(f"x = {x}, prédiction = { Zpredict(x)}")
```

Expliquer pourquoi les résultats ne sont pas ceux attendus ? _____

```
1] ✓ 0.0s  
x = 32.0, prédiction = 0  
x = 85.4, prédiction = 0  
x = 73.0, prédiction = 0
```

Exercice 4 : Calcul de l'erreur

Dans notre cas, notre modèle utilise une fonction de seuil elle n'est pas dérivable donc on utilise une formule simple

erreur = y_attendu - y_pred

avant de créer cette partie d'algorithme nous avons besoin d'un outil qu'on appelle en python `zip()`

La fonction intégrée **zip()** permet de **regrouper plusieurs listes (ou itérables) élément par élément**.

1) Le Zip : Dans la fenêtre interactive tester cela :

```
noms = ["Alice", "Bob", "Charlie"]  
ages = [25, 30, 22]  
zipped = zip(noms, ages)  
print(list(zipped))
```



Chaque élément de sortie est un **tuple** contenant un élément de chaque liste à la même position.

Voici la boucle que l'on peut faire :

```
for nom, age in zip(noms, ages):  
    print(f"{nom} a {age} ans")
```



```
Alice a 25 ans  
Bob a 30 ans  
Charlie a 22 ans
```

Si les listes n'ont **pas la même taille**, `zip()` s'arrête à la **plus courte**.

On peut aussi `dezipper`

```

z = zip(noms, ages)
n, a = zip(*z)
print(n) # ("Alice", "Bob", "Charlie")
print(a) # (25, 30, 22)

```

2) Fait une boucle d'un zip des entrée X et sortie Y

attention normalement X et Y on déjà été défini comme cela :

```

X = dataframe1MonJeuDeDonneeDentrainement["vitesse"].tolist()
Y = dataframe1MonJeuDeDonneeDentrainement["danger"].tolist()

```

Voici l'algo :

Pour chaque **petit x** , **petit y** dans le zip (grand X et grand Y) :

La prédiction y = appel de la fonction Zpredict pour **petit x**

Erreur : petit y moins **La Prédiction y**

Afficher **petit x** **petit y** , et **La prédiction y** ainsi que l' **Erreur**.

Reprenez cet algo et fait le code en python dans votre fichier TP4.py.

✓ `import pandas as pandasLib ...`

```

x = 32.0, y = 0, y_pred = 0, erreur = 0
x = 85.4, y = 1, y_pred = 0, erreur = 1
x = 73.0, y = 0, y_pred = 0, erreur = 0
x = 91.0, y = 1, y_pred = 0, erreur = 1
x = 36.2, y = 0, y_pred = 0, erreur = 0
x = 81.0, y = 1, y_pred = 0, erreur = 1
x = 69.0, y = 0, y_pred = 0, erreur = 0
x = 99.2, y = 1, y_pred = 0, erreur = 1
x = 32.9, y = 0, y_pred = 0, erreur = 0
x = 85.1, y = 1, y_pred = 0, erreur = 1
x = 53.0, y = 0, y_pred = 0, erreur = 0
x = 91.0, y = 1, y_pred = 0, erreur = 1
x = 36.3, y = 0, y_pred = 0, erreur = 0
x = 81.0, y = 1, y_pred = 0, erreur = 1
x = 79.0, y = 1, y_pred = 0, erreur = 1
x = 99.0, y = 1, y_pred = 0, erreur = 1

```

Analyser la sortie et vérifier que l'erreur est bien positionnée a 1 quand la vitesse x et le danger y_prédit ne sont pas en adéquation . C'est-à-dire qui ne sont pas correct .

En résumé :

Votre algo détecte maintenant correctement les erreurs du neurone qui l'entraîne, on va donc utiliser ces erreurs détectées pour corriger nos paramètres de neurone c'est-à-dire adapter

w1 et le **b** au fur et à mesure de l'entrainement